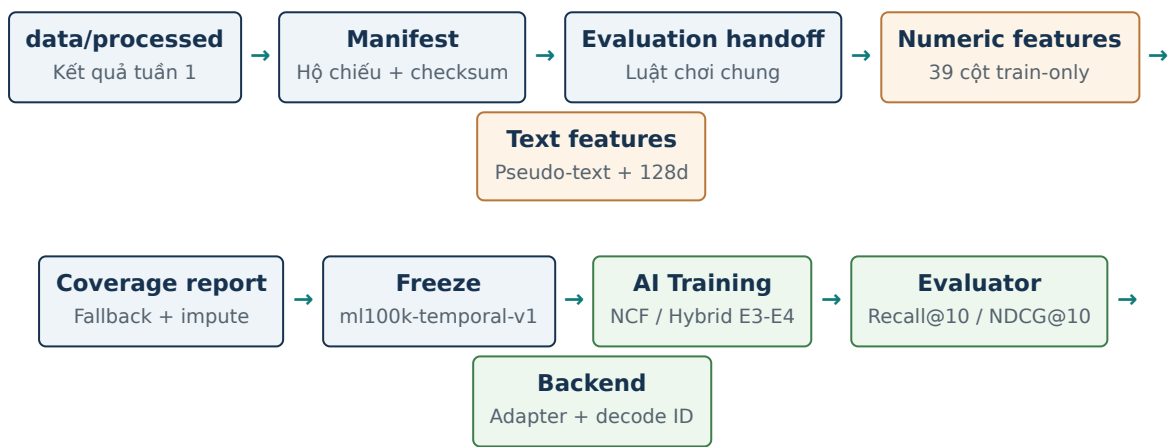


CẨM NANG DATA TUẦN TĂNG TỐC

Từ dữ liệu model-ready đến hợp đồng feature 167 chiều
cho Hybrid NCF, evaluator và Backend



Nguyên tắc xuyên suốt: mọi thống kê chỉ fit từ train; mọi artifact tái tạo được từng byte; mọi con số có nguồn kiểm chứng.

Đối tượng	Nội dung
Người phụ trách Data	Hiểu 5 tầng xử lý mới, giải thích được từng con số khi phản biện
Thành viên AI	Biết cách load artifact, quy tắc 167 chiều và ba điều cấm
Backend	Biết luồng dữ liệu lúc chạy thật và trường nào phải lưu theo run

Mục lục

1. Tuần tăng tốc giải quyết vấn đề gì?	3
2. Bẫy khái niệm mới cần nắm chắc	3
3. Tầng 1 — data_manifest.json: hệ chiếu của dữ liệu	4
4. Tầng 2 — Evaluation handoff: luật chơi chung	5
5. Tầng 3 — Numeric features: 39 cột không leakage	6
6. Tầng 4 — Text features: pseudo-text và tương tác 128 chiều	7
7. Hợp đồng 167 chiều — cách Hybrid NCF ăn dữ liệu	9
8. Tầng 5 — Coverage report	9
9. Test là bằng chứng, không phải thủ tục	10
10. Đường đi của dữ liệu lúc chạy thật (serving)	10
11. Runbook — chạy lại toàn bộ từ clone sạch	11
12. Freeze và versioning	11
13. Bàn giao — ai nhận gì, xác nhận thế nào	11
14. Câu hỏi phản biện có thể gặp — và cách trả lời	11
15. Cheat sheet một trang	13
16. Thuật ngữ Việt - Anh	13
17. Kết luận	14

1. Tuần tăng tốc giải quyết vấn đề gì?

Cuối tuần 1, phần Data dừng ở năm artifact model-ready: `train.csv`, `validation.csv`, `test.csv`, `movies.csv` và `id_mappings.json`. Chúng sạch, chia đúng thời gian và không leakage — nhưng mô hình chưa "ăn" trực tiếp được. Còn bốn khoảng trống:

- **Chưa có bằng chứng tái lập.** Ai dám chắc file trên máy Thành giống hệt file trên máy mình? Cần một "hệ chiếu" ghi checksum.
- **Chưa có luật chơi chung cho đánh giá.** Nếu mỗi model tự chọn candidate và negative thì Recall@10 không so được với nhau. Cần đóng gói luật đó thành file.
- **Hybrid NCF cần feature thật.** Tuần 1 model chỉ có embedding từ ID. Hybrid cần genre, năm phát hành, lịch sử sở thích và text — tất cả phải tính mà không nhìn trộm validation/test.
- **MovieLens không có text.** Không có câu sở thích của user, không có overview phim. Muốn trả lời RQ2 ("text có giúp gì không?") phải tự sinh text một cách trung thực và có kiểm soát.

Tuần tăng tốc lấp bốn khoảng trống đó bằng năm tầng xử lý mới, mỗi tầng một script, một module, một bộ test và một artifact đầu ra:

Ngày	Commit	Script → Artifact	Trả lời câu hỏi
1	c76b2fc manifest	create_data_manifest.py → data_manifest.json	Dữ liệu này là phiên bản nào, có đúng từng byte không?
2	a26a6fc handoff	prepare_evaluation_data.py → 4 file trong outputs/evaluation/	Mọi model so nhau bằng luật gì?
3	c640615 numeric	prepare_numeric_features.py → 3 file numeric	Hybrid lấy genre/năm/lịch sử ở đâu mà không leakage?
4	d2dc604 text	prepare_text_features.py → 5 file text	Không có text thật thì nhánh text học từ đâu?
5	0a49bb7 guards	report_feature_coverage.py → coverage report + 13 test	Chứng minh không leakage và minh bạch fallback thế nào?
6-7	348f32c ... 91fd091	README, docs, freeze, checklist, freeze-guard test	Bàn giao và khoá phiên bản ra sao?

Cách đọc tài liệu này: mỗi tầng có bốn phần — *vấn đề*, *cách hoạt động từng bước*, *ví dụ tính tay bằng số thật*, và *AI/Backend dùng nó thế nào*. Mọi con số đều tái tạo được bằng lệnh trong mục Runbook (mục 11).

2. Bảy khái niệm mới cần nắm chắc

Artifact. Một file đầu ra có phiên bản, sinh ra bằng script, không sửa tay. Nguyên tắc của nhóm: artifact không commit lên Git (nặng, sinh lại được) — thứ được commit là *code sinh ra nó* và *con số kiểm chứng nó*.

Checksum SHA-256. "Dấu vân tay" 64 ký tự của một file: đổi 1 byte là đổi cả chuỗi. Hai máy chạy pipeline ra cùng checksum nghĩa là dữ liệu giống nhau *từng byte*, không phải "trông giống".

Data version và feature contract. `ml100k-temporal-v1` đặt tên cho *cách chia dữ liệu*; `hybrid-v1-167` đặt tên cho *cách xếp feature*. Checkpoint và kết quả thí nghiệm luôn ghi kèm hai tên này — đổi một trong hai mà không đổi tên là phá tính so sánh được.

Fit và transform. "Fit" là *học tham số* từ dữ liệu (median, mean, độ lệch chuẩn); "transform" là *áp dụng* tham số đã học lên bất kỳ dữ liệu nào. Luật sắt: chỉ fit trên train; validation/test/phim mới chỉ được transform. Đây là dạng tổng quát của bài học leakage tuần 1.

Deterministic + seed. Chạy hai lần ra kết quả giống hệt. Chỗ nào có "ngẫu nhiên" (chọn mẫu câu pseudo-text) thì ngẫu nhiên đó được điều khiển bằng seed ghi trong config — nên vẫn deterministic.

Tích Hadamard (⊙). Nhân hai vector cùng chiều theo *từng phần tử*: $[a,b] \odot [c,d] = [ac, bd]$. Khác tích vô hướng (ra một số), Hadamard ra vector cùng chiều — giữ được kích thước hợp đồng.

Fallback. Nhánh dự phòng khi quy tắc chính không áp dụng được, luôn kèm một cờ (`used_fallback`) để đếm và báo cáo. Fallback không phải lỗi — giấu fallback mới là lỗi.

3. Tầng 1 — data_manifest.json: hệ chiếu của dữ liệu

Vấn đề

Bảy người, bảy máy, một dataset. Khi bằng kết quả của Thành khác của Công Thành, câu đầu tiên phải trả lời được là: "hai người có đang dùng đúng cùng một dữ liệu không?". Trước tuần này, câu đó chỉ trả lời được bằng niềm tin.

Cách hoạt động

scripts/create_data_manifest.py (logic trong src/cinematch/data/manifest.py) đọc lại toàn bộ artifact tuần 1, **tự đếm và tự băm** — không tin bất kỳ con số gõ tay nào — rồi ghi một file JSON gồm sáu khối:

Khối	Ghi gì	Vì sao cần
dataset	Tên, version: ml100k-temporal-v1, thang rating [1, 5]	Định danh phiên bản dữ liệu
entities	100,000 rating, 943 user, 1,682 phim	Khớp với con số kỳ vọng trong config
split	Chiến lược per-user temporal, tỷ lệ, số dòng 80,014 / 10,132 / 9,854	Chốt cách chia, không ai chia lại khác đi
quality	Overlap = 0, temporal violation = 0, cold-start giữ nguyên	Bằng chứng audit tuần 1 vẫn đúng
evaluation_contract	Positive ≥ 4.0, K = 10, 100 negative, seed 42	Tham số đánh giá chỉ có một nguồn
feature_contract	hybrid-v1-167 + thứ tự block: genres 19, year 1, history 19, text 128	Hợp đồng feature cho Hybrid (mục 7)
artifacts	Đường dẫn, số byte, số dòng và SHA-256 của 7 file	Kiểm chứng từng byte giữa các máy

Cấu trúc file được khoá bằng JSON Schema (schemas/data_manifest.schema.json): thiếu trường, sai kiểu là test test_data_manifest.py đổ.

Ví dụ đọc manifest

```
"artifacts": {
  "raw_ratings": { "path": "data/raw/ml-100k/u.data",
    "bytes": 1979173, "rows": 100000,
    "sha256": "06416e597f82b734..." } , ... }
```

Muốn biết máy mình và máy bạn cùng dữ liệu: cả hai chạy pipeline, mở manifest, so 7 dòng sha256. Trùng cả 7 là xong — đây chính là nghi thức xác nhận trong tin bàn giao ngày 6.

AI và Backend dùng thế nào

AI: mỗi checkpoint lưu kèm data_version + feature_contract_version đọc từ manifest. Khi tuần 3 chạy ma trận 3 seed, mọi run đều truy ngược được về đúng dữ liệu. **Backend:** theo quy tắc BR-09 của spec, mỗi recommendation run phải lưu model version, strategy, threshold — cột data_version lấy từ đây, để một kết quả demo bất kỳ đều giải trình được "sinh ra từ dữ liệu nào, model nào".

Hỏi nhanh: vì sao manifest tự đếm thay vì chép số từ config? — Vì config ghi kỳ vọng, manifest ghi thực tế. Khi hai bên lệch nhau, test nổ và đó chính là lúc phát hiện dữ liệu hỏng.

4. Tầng 2 — Evaluation handoff: luật chơi chung cho mọi model

Vấn đề

Recall@10 chỉ có nghĩa khi mọi model trả lời *cùng một đề bài*. Đề bài gồm ba câu hỏi: với mỗi user, (a) những phim nào coi như "đã xem" và không được tính điểm? (b) những phim nào là "đáp án đúng"? (c) chấm trên tập ứng viên nào? Nếu để mỗi model tự trả lời, so sánh vô nghĩa. Tầng này đóng gói đáp án chung thành file để không ai "tự chế luật".

Cách hoạt động từng bước

`prepare_evaluation_data.py` (logic trong `evaluation_handoff.py`) đọc ba partition và sinh bốn file trong `outputs/evaluation/`:

File	Nội dung
<code>catalog.json</code>	1,682 movie_index hợp lệ, đánh dấu 1,611 phim đã thấy trong train và 71 phim cold-start
<code>seen_items.json</code>	Với từng user: <code>seen</code> = phim trong train u validation
<code>positive_test_items.json</code>	Với từng user: phim trong test có rating ≥ 4.0
<code>evaluation_data_summary.json</code>	Toàn bộ tham số protocol + con số 943 / 836 / 107

Ba quy tắc đằng sau, mỗi quy tắc có lý do:

- **Seen = train u validation.** Model được phép "biết" cả hai (train để học, validation để chọn checkpoint), nên khi chấm điểm trên test, gợi ý lại phim user đã tương tác trong hai tập đó không được tính là sai — chúng bị loại khỏi bể negative.
- **Positive = test rating ≥ 4.0 .** Ngưỡng 4.0 nằm trong config, dùng chung toàn dự án (trùng ngưỡng pseudo-text — cùng một định nghĩa "thích").
- **836 evaluable, 107 skipped.** 107 user không có phim nào ≥ 4.0 trong test — không có "đáp án đúng" thì không chấm Recall được. Họ không bị xóa, chỉ được đánh dấu skip, và **mọi model phải có cùng cặp số 836/107**; lệch là protocol sai.

Ví dụ: user 1

```
seen(user 1)  = 245 phim (218 train + 27 validation)
positive(user 1) = 18 phim test có rating  $\geq 4.0$ 
candidate(user 1) = 18 positive + 100 negative
                  (bốc từ 1682 - seen - positive, seed 42)
Đề bài cho model: xếp hạng 118 ứng viên; đếm bao nhiêu
trong 18 đáp án lọt top 10 → Recall@10, NDCG@10.
```

AI và Backend dùng thế nào

Thành (evaluator) là khách hàng chính: script `evaluate_topk.py` load ba file này, dựng candidate và chấm *mọi* predictor (MP, MF, GMF, NCF, Hybrid) trên cùng đề bài — bảng RQ1 vì thế mới hợp lệ. **Son** dùng summary để báo cáo evaluated/skipped. **Backend** không dùng trực tiếp lúc chạy thật (phục vụ người dùng thì gợi ý trên cả catalog sau khi lọc ràng buộc), nhưng dùng `catalog.json` khi cần biết phim nào cold-start để chọn đường dự đoán phù hợp.

Hỏi nhanh: tại sao không loại 71 phim cold-start cho "sạch"? — Vì cold-start là hạn chế thật của collaborative filtering, không phải dữ liệu bẩn. Giữ lại và báo cáo riêng warm-start mới là trung thực; xóa đi là làm đẹp số liệu.

5. Tầng 3 — Numeric features: 39 cột không leakage

Vấn đề

NCF thuần chỉ nhìn ID — user 405 là "user 405", không biết họ thích gì. Hybrid muốn model nhìn thêm *nội dung*: phim này thể loại gì, ra năm nào, user này trước giờ chấm thể loại nào cao. Nhưng năm phát hành có giá trị từ 1922 đến 1998 còn genre là 0/1 — phải chuẩn hoá, và chuẩn hoá là chỗ leakage dễ chui vào nhất.

Phía phim: 19 genre + 1 năm chuẩn hoá

Genre lấy thẳng từ `movies.csv` (multi-hot 0/1, một phim có thể nhiều genre). Năm phát hành đi qua hai bước, **cả hai chỉ fit trên 1,611 phim xuất hiện trong train**:

Bước fit (train-only):
 median = 1995.0 (điển cho phim thiếu năm)
 mean = 1989.3911 (sau khi điển)
 std = 14.1825 (population, ddof=0)
 Bước transform (mọi phim): $z = (\text{year} - \text{mean}) / \text{std}$

Ví dụ tính tay bằng số thật:

Phim	Năm	Tính	z
Toy Story (1995)	1995	$(1995 - 1989.39) / 14.18$	+0.395
Citizen Kane (1941)	1941	$(1941 - 1989.39) / 14.18$	-3.412
Nosferatu (1922)	1922	$(1922 - 1989.39) / 14.18$	-4.752
Phim thiếu năm (1 phim)	→ 1995	điển median rồi tính như trên	+0.395

$z = -4.75$ của Nosferatu trông "to" nhưng là tín hiệu thật (phim rất cũ) — chính sách là **báo cáo, không cắt** (mục 8). 71 phim không có trong train được transform bằng tham số đã fit, tuyệt đối không fit lại.

Phía user: 19 cột lịch sử thể loại

Với mỗi user và mỗi genre, lấy các rating **trong train** cho phim thuộc genre đó, quy tâm rồi lấy trung bình:

$\text{score}(\text{user}, \text{genre}) = \text{mean}((\text{rating} - 3) / 2) \in [-1, +1]$
 rating 5 → +1.0 rating 4 → +0.5 rating 3 → 0
 rating 2 → -0.5 rating 1 → -1.0 chưa xem genre → 0

Vì sao quy tâm quanh 3? Vì rating 3 là "thường thường" — nếu lấy trung bình thô, một user khó tính chấm toàn 3 sẽ trông giống "thích mọi thứ mức 0.6". Quy tâm làm 0 nghĩa là *trung lập*, dấu +/- nghĩa là thích/không thích. Ví dụ user 1 (218 rating train): history_Documentary = +1.0 (3 phim, toàn 5 sao), history_Sci-Fi = +0.486 (36 phim), history_Drama = +0.470 (82 phim).

Đầu ra và cách dùng

Ba artifact: `movie_numeric_features.csv` (1,682 dòng × 20 cột), `user_genre_profiles.csv` (943 × 19), `numeric_feature_preprocessor.json` (median/mean/std, thứ tự cột `ordered_feature_columns`, `fit_partition`). Hàm `build_interaction_numeric_features(interactions, movie_features, user_profiles)` join theo ID và trả tensor `[batch, 39]` — join thiếu là raise, không âm thầm điển bù.

AI: 39 cột này là ba block đầu của hợp đồng 167 (mục 7); ablation E3 dùng đúng chúng. Đây cũng là nền cho cold-start: user demo mới rate 5-10 phim là tính được ngay history profile bằng *đúng công thức trên*, không cần train lại. **Backend:** không đọc file CSV này trực tiếp — adapter chỉ đưa `user_index/movie_index`, việc join do hàm builder làm; điều Backend cần nhớ là *công thức history dùng được cho user mới ở luồng onboarding*.

6. Tầng 4 — Text features: pseudo-text và tương tác 128 chiều

Vấn đề, và tuyên bố trung thực

MovieLens 100K không có câu sở thích của user, cũng không có overview phim. Nếu nhánh text của Hybrid nhận vector 0 lúc train thì nó không học được gì và ablation E3-với-E4 vô nghĩa. Giải pháp: **tự sinh text tiếng Anh một cách có kiểm soát từ chính rating train** — và nói thẳng trong mọi tài liệu rằng đây là text tổng hợp, không phải ngôn ngữ người dùng thật. RQ2 vì thế chỉ được kết luận thận trọng.

Quy tắc sinh pseudo-text cho user — từng bước

1. Join train.csv với movies.csv theo movie_id
2. Với mỗi genre: tính mean rating và số phim đã chấm
3. Loại "unknown" (không phải sở thích ngữ nghĩa)
4. GIỮ genre có: mean ≥ 4.0 VÀ count ≥ 3
5. Xếp: mean giảm dần $\rightarrow$ count giảm dần $\rightarrow$ thứ tự genre cố định
6. Lấy tối đa 3 genre
7. Không genre nào đạt $\rightarrow$ fallback: lấy genre mean cao nhất (bỏ điều kiện count), bắt cờ used_fallback
8. Chọn 1 trong 3 mẫu câu bằng (seed + user_id) $\rightarrow$ câu tiếng Anh

Ví dụ tính tay — user 1 (218 phim train):

Genre	Mean	Count	Kết quả với luật "mean ≥ 4.0 và count ≥ 3 "
Documentary	5.000	3	ĐẠT — đúng chạm ngưỡng count
Film-Noir	5.000	1	Loại — mean tuyệt đối nhưng chỉ 1 phim, không đủ bằng chứng
Sci-Fi	3.972	36	Loại — nhiều quan sát nhưng mean dưới 4.0
Drama	3.939	82	Loại — tương tự

$\rightarrow$ preferred_genres = "Documentary", used_fallback = False
 $\rightarrow$ pseudo_text = "I usually choose films in Documentary."

Trường hợp Film-Noir chính là lý do luật count ≥ 3 ra đời ở ngày 5: bản đầu (chỉ cần mean ≥ 4.0) cho War/Film-Noir/Crime thống trị top genre toàn dự án vì hiệu ứng mẫu nhỏ — một phim 5 sao là genre hiếm "thắng". Sau khi thêm điều kiện count, Film-Noir tụt từ 264 xuống 157 user, Drama và Romance (nhiều quan sát) vào top; đổi lại fallback tăng từ 85 lên **235/943 user (24.92%)** — con số này được báo cáo công khai, không giấu.

Text phía phim và mã hoá

Mỗi phim một câu từ dữ liệu có thật trong catalog, không bịa: "Toy Story (1995). Genres: Animation, Children and Comedy." Cả câu user lẫn câu phim đi qua cùng một PreferenceTextEncoder (signed feature hashing 128 chiều, chuẩn hoá L2 — đã học ở cẩm nang tuần 1): user ra ma trận $[943 \times 128]$, phim ra $[1682 \times 128]$.

Tích Hadamard — vì sao và hoạt động ra sao

Với một cặp (user, phim): $\text{text_interaction} = u \odot m$ (nhân từng phần tử, vẫn 128 chiều). Ví dụ đồ chơi 4 chiều:

$u = [0.8, 0.0, -0.5, 0.2]$ (user hay nói về "action")
 $m = [0.9, 0.4, 0.0, 0.1]$ (phim có chữ "action")
 $u \odot m = [0.72, 0.0, 0.0, 0.02] \rightarrow$ phần tử lớn ở đúng chỗ
 hai bên CÙNG có tín hiệu

Hashing đặt từ giống nhau vào cùng vị trí, nên phần tử lớn của $u \odot m$ nghĩa là "câu của user và câu của phim chia sẻ từ/cụm từ". So với hai lựa chọn khác: *concat* (u nối m) làm phồng hợp đồng lên 256 chiều và bắt MLP tự học phép so khớp; *cosine* (một số duy nhất) nén quá mạnh, mất thông tin "giống nhau ở chủ đề nào". Hadamard giữ đúng 128 chiều — hợp đồng 167 không đổi — và cho MLP biết giống nhau ở đâu.

AI và Backend dùng thế nào

AI: E3 (mask nhánh text = 0) so với E4 (có text) trên cùng split/candidate/seed → câu trả lời RQ2. Load bằng `load_text_feature_artifacts` + `build_interaction_text_features`; **cắm sinh lại pseudo-text trong lúc train**. **Backend:** đây là chỗ dữ liệu chạm sản phẩm — khi user thật gõ câu sở thích tiếng Anh lúc onboarding, câu đó đi qua *đúng encoder này* (cùng artifact, cùng version), Hadamard với vector phim có sẵn, và Hybrid dự đoán được ngay cả khi user không có embedding ID. Đó là đường cold-start "Hybrid-only" trong kế hoạch.

7. Hợp đồng 167 chiều — cách Hybrid NCF "ăn" dữ liệu

Mọi tầng ở trên hội tụ về một vector side-feature cố định cho từng cặp (user, phim), nối vào sau hai embedding ID:

32 user embedding (học từ user_index)	32 movie embedding (học từ movie_index)	19 movie genres multi-hot	1 year z-score train	19 user history centered $[-1,1]$	128 text interaction $u \odot m$
--	--	--	-----------------------------------	--	---

Side features = 19 + 1 + 19 + 128 = **167** → cùng 64 chiều embedding vào MLP (64-32-16) → điểm dự đoán 1-5.

Thứ tự bốn block là **hợp đồng cứng**, ghi ở ba nơi phải luôn khớp nhau: hằng số trong `hybrid_features.py`, khối `feature_contract.ordered_layout` trong `manifest`, và `ordered_feature_columns` trong `preprocessor` JSON. Hàm `build_hybrid_side_features(genres, year, history, text)` validate shape từng block rồi mới concat — đưa sai thứ tự hay sai chiều là raise ngay.

Ba điều cấm khi tiêu thụ (đã ghi trong tin bàn giao)

- **Không hardcode slice** kiểu `[ :19 ]`, `[ 20:39 ]` — cắt block theo `ordered_feature_columns` đọc từ `preprocessor`, để nếu hợp đồng đổi version thì code đọc nổ thay vì âm thầm lệch cột.
- **Không tự sinh lại feature trong training** — load artifact. Sinh lại nghĩa là có hai nguồn sự thật, và một trong hai sẽ sai lúc nào không biết.
- **Checkpoint phải lưu version của cả hai preprocessor** (numeric + text) cùng data version — để tuần 3 mọi con số trong báo cáo truy ngược được.

Ablation dùng **feature mask**: E3 = mask block text về 0, E4 = giữ nguyên — cùng một class model, cùng API, chỉ khác mask, nên khác biệt đo được là do feature chứ không do kiến trúc.

8. Tầng 5 — Coverage report: nhìn thấy những gì thường bị giấu

`report_feature_coverage.py` đọc lại các artifact (không sửa gì) và trả lời câu "feature phủ được bao nhiêu, chỗ nào phải dùng đường dự phòng?" — những thứ mà nếu không đo sẽ chỉ lộ ra khi bị phản biện hỏi:

Chỉ số	Giá trị	Đọc thế nào
User dùng fallback pseudo-text	235 / 943 (24.92%)	1/4 user không có genre đạt cả hai điều kiện — chấp nhận được vì fallback vẫn có nghĩa và có cờ đánh dấu
User có history toàn 0	0	Mọi user đều có tín hiệu lịch sử (ai cũng ≥ 20 rating)
Phim thiếu năm (điểm median)	1	Chỉ 1/1,682 — không đáng lo
Phim ngoài train (transform bằng tham số fit sẵn)	71	Trùng số cold-start của evaluation — hai tầng nhất quán
Khoảng year chuẩn hoá	$[-4.75, 0.61]$	-4.75 là Nosferatu 1922 — tín hiệu thật
Year vượt ngưỡng cảnh báo $ z > 6$	0	Không có giá trị bất thường cần điều tra
Vector text chuẩn L2 = 1	100%	Encoder hoạt động đúng trên toàn bộ 2,625 câu

Chính sách outlier: báo cáo, không cắt (clip). Cắt z về ± 3 sẽ làm Citizen Kane (-3.41) và Nosferatu (-4.75) trông "cùng độ cũ" — bóp méo một tín hiệu thật để đổi lấy bảng số đẹp. Nếu sau này cần cắt, đó phải là quyết định có ghi chép và bump version, không phải mặc định âm thầm.

9. Test là bằng chứng, không phải thủ tục

Toàn dự án hiện có **257 test**; phần Data tuần tăng tốc đóng góp ~60. Chúng chia sáu nhóm, mỗi nhóm chặn một kiểu sai khác nhau — khi phản biện hỏi "làm sao em biết không leakage?", câu trả lời là bằng này:

Nhóm test	Ví dụ tiêu biểu	Chặn kiểu sai nào
Unit — công thức	History dùng mean của $(r-3)/2$; year diễn median trước khi scale	Code lệch khỏi công thức đã công bố
Leakage — nhiễu tương lai	Đổi metadata của phim chỉ có ở tương lai → scaler và profile <i>không đổi</i>	Thống kê fit lẫn dữ liệu ngoài train
Script wiring	Đọc source hai script feature, khẳng định không tham chiếu <code>paths.validation / paths.test</code>	Builder sạch nhưng script gọi sai
Tái tính độc lập trên dữ liệu thật	Tính lại history của user nặng nhất bằng pandas thuần, so với artifact	Fixture đúng nhưng dữ liệu thật sai
Round-trip + contract	Save rồi load artifact ra đúng nội dung; vector đảo thứ tự bị từ chối; hợp đồng 167 khớp	Artifact hỏng âm thầm giữa hai người
Freeze guard	<code>test_default_data_config_is_valid</code> pin data version, 167, ngưỡng 4.0, K=10, seed, min-count 3	Ai đó sửa config mà không bump version

Ý tưởng đáng nhớ nhất cho phần trình bày: **tính chất "không leakage" không phải lời hứa mà là thứ CI cưỡng chế** — muốn vi phạm phải làm đổ ít nhất một test và qua được review.

10. Đường đi của dữ liệu lúc chạy thật (serving)

Đây là phần trả lời "đồng artifact này giúp gì cho Backend?". Lúc phục vụ người dùng thật, dữ liệu đi theo hai luồng:

User cũ (có trong MovieLens — dùng cho thí nghiệm)

Backend (movie_id) → id_mappings → movie_index
→ predictor (NCF/Hybrid): embedding + numeric + text artifact
→ điểm 1-5 cho từng thành viên × từng phim
→ group aggregation (Sơ) → Top 10 → id_mappings → movie_id → API

User demo mới (không có trong MovieLens — dùng cho bảo vệ)

Onboarding: rate 5-10 phim + gõ câu sở thích tiếng Anh
1. History 19 chiều ← tính bằng ĐÚNG công thức mục 5 từ 5-10 rating vừa nhập
2. Text 128 chiều ← câu thật đi qua ĐÚNG encoder mục 6, Hadamard với vector phim có sẵn
3. Đường Hybrid-only: user embedding = 0, dựa vào (1)+(2) (song song: fold-in embedding trung bình lảng giềng — việc của Thành)

Điểm mấu chốt: **không có bước train nào lúc serving**. Mọi thứ user mới cần — công thức history, encoder, vector phim, tham số scaler — đều là artifact đã đóng băng. Đó là lý do các preprocessor JSON tồn tại: chúng là "bộ nhớ" để lúc chạy thật tái hiện chính xác phép biến đổi lúc train. Backend chỉ cần gọi adapter với ID; nếu model/artifact lỗi, adapter trả lỗi rõ ràng và demo rơi về MF theo kế hoạch dự phòng.

11. Runbook — chạy lại toàn bộ từ clone sạch

```
git clone -b feature/data-hai-anh <repo> && cd CinematchAi
python3.12 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"

python scripts/download_data.py          # tải ml-100k
python scripts/prepare_data.py           # split + mapping (tuần 1)
python scripts/audit_splits.py           # audit không leakage
python scripts/create_data_manifest.py    # tầng 1
python scripts/prepare_evaluation_data.py # tầng 2
python scripts/prepare_numeric_features.py # tầng 3
python scripts/prepare_text_features.py  # tầng 4
python scripts/report_feature_coverage.py # tầng 5
python -m pytest -q                      # 257 passed
```

Số chuẩn để đối chiếu (sai bất kỳ số nào = dừng lại điều tra)

Điểm kiểm tra	Giá trị chuẩn
Split	80,014 / 10,132 / 9,854 — tổng đúng 100,000; overlap 0; temporal violation 0
Manifest	7 artifact có sha256; trùng từng chuỗi giữa các máy
Evaluation	943 user; 836 evaluable; 107 skipped; K=10; 100 negative; seed 42
Numeric	median 1995.0; mean 1989.3911; std 14.1825; fit trên 1,611 phim; 39 cột
Text	943 câu user; 1,682 câu phim; fallback 235 (24.92%); vector [943×128], [1682×128] chuẩn L2
Coverage	history rỗng 0; year range [−4.75, 0.61]; outlier z >6: 0
Test	257 passed

12. Freeze và versioning — vì sao "khóa" lại quan trọng

Cuối ngày 7, `ml100k-temporal-v1` và `hybrid-v1-167` được tuyên bố đóng băng (DATA_REPORT mục 16). Nghĩa là: split, schema, mapping, layout 39 cột và block text 128 chiều **không được đổi nữa** — vì mọi checkpoint Công Thành sắp train và mọi bảng số Thành sắp chạy đều tham chiếu hai cái tên này. Đối dữ liệu sau khi có kết quả = mọi kết quả cũ thành rác không so sánh được.

Muốn đổi thì làm công khai: bump `data.version` trong config (ví dụ `-v2`), sinh lại manifest, được nhóm đồng ý, và chấp nhận chạy lại thí nghiệm. Cơ chế cường chế gồm hai lớp: **freeze-guard test** pin từng giá trị trong config (đổi mà không sửa test là CI đỏ), và **checksum trong manifest** (dữ liệu lệch là bàn giao phát hiện ngay).

13. Bàn giao — ai nhận gì, xác nhận thế nào

Người nhận	Artifact chính	Việc họ làm với nó	Xác nhận bàn giao
Thành	outputs/evaluation/*, manifest	Dựng candidate chung, chấm Top-K mọi model; fold-in cold-start	Chạy runbook từ clone sạch, trả lời bằng số pytest + dòng sha256 đầu tiên của manifest; tick vào checklist DATA_REPORT mục 16
Công Thành	outputs/features/* (8 file), loader	Train Hybrid E3/E4 theo hợp đồng 167; tuân ba điều cấm mục 7	
Chúc	id_mappings.json, manifest	Decode index↔ID trong adapter; lưu data_version theo run	

14. Câu hỏi phản biện có thể gặp — và cách trả lời

Manifest để làm gì, config chưa đủ sao? — Config ghi kỳ vọng; manifest tự đếm và tự bấm thực tế. So hai bên là cách phát hiện dữ liệu hỏng; so manifest giữa hai máy là cách chứng minh tái lập từng byte.

Vì sao 107 user bị bỏ khi đánh giá? — Họ không có phim test nào ≥ 4.0 , tức không có "đáp án" để chấm Recall. Bỏ có kiểm soát và báo cáo công khai; giữ họ với đáp án rỗng chỉ tạo số 0 giả làm loãng trung bình. Mọi model dùng cùng danh sách 836/107 nên vẫn công bằng.

Seen items gồm cả validation — có phải leakage không? — Không. Leakage là để thông tin tương lai lọt vào *lúc học*. Seen chỉ quyết định phim nào không bị đem ra làm negative *lúc chấm*; model được phép biết validation vì đã dùng nó chọn checkpoint.

Vì sao year scaler chỉ fit trên 1,611 phim? — 71 phim còn lại chỉ xuất hiện ở validation/test. Đưa chúng vào mean/std là để tương lai rò vào tham số học — đúng định nghĩa leakage. Có test đổi năm của một phim tương lai thành 9999 và khẳng định scaler không đổi.

Vì sao cần count ≥ 3 trong pseudo-text? — Chống bias mẫu nhỏ. Ví dụ thật: user 1 có Film-Noir mean 5.0 nhưng từ đúng 1 phim — một quan sát không phải bằng chứng sở thích. Trước khi thêm luật, các genre hiếm thống trị top toàn dự án; sau khi thêm, phân bố hợp lý hơn, đổi bằng fallback tăng lên 235 user và được báo cáo.

Pseudo-text có phải "gian lận" không — text sinh từ chính rating? — Không giấu điều đó: tài liệu tuyên bố rõ đây là feature tổng hợp từ train, không phải ngôn ngữ độc lập. Vì thế RQ2 chỉ kết luận "nhánh text mang thêm tín hiệu gì trên protocol này", không tuyên bố "text người thật sẽ giúp". Giá trị thật của nhánh text nằm ở serving: user thật gõ câu thật, đi qua đúng pipeline này.

Vì sao Hadamard mà không phải concat hay cosine? — Concat phồng hợp đồng lên 256 chiều và đẩy việc so khớp cho MLP; cosine nén còn 1 số, mất "giống ở chủ đề nào". Hadamard giữ 128 chiều đúng hợp đồng và cho biết vị trí trùng tín hiệu.

Nếu thay hashing bằng Sentence Transformer thì phải đổi những gì? — Encoder mới phải ra 128 chiều (hoặc chiều về 128), bump version trong text preprocessor, sinh lại hai ma trận vector, giữ nguyên Hadamard và hợp đồng 167. Mọi checkpoint cũ vẫn giải trình được vì đã ghi version encoder cũ.

Đóng băng rồi phát hiện lỗi dữ liệu thì sao? — Bump version, sửa, sinh lại manifest, chạy lại thí nghiệm bị ảnh hưởng. Đắt, nhưng rẻ hơn việc âm thầm sửa và có hai bảng số không so được với nhau.

Fallback 24.92% có quá cao không? — Đó là cái giá của tiêu chí bằng chứng chặt trên dataset nhỏ. Fallback vẫn cho câu có nghĩa (genre tốt nhất user từng chấm), có cờ riêng, và evaluator có thể phân tích riêng nhóm này. Cao-nhưng-minh-bạch tốt hơn thấp-nhưng-bị.

15. Cheat sheet một trang

Script (chạy theo thứ tự)	Artifact sinh ra	Ai dùng
create_data_manifest.py	data_manifest.json	Cả nhóm (checksum), Chúc (data_version theo run)
prepare_evaluation_data.py	catalog / seen_items / positive_test_items / summary.json	Thành (evaluator), Sơn (báo cáo 836/107)
prepare_numeric_features.py	movie_numeric_features.csv , user_genre_profiles.csv , numeric_feature_preprocessor.json	Công Thành (Hybrid E3), Backend (công thức cold-start)
prepare_text_features.py	user_pseudo_text.csv , movie_text.csv , 2 file .npz , text_feature_preprocessor.json	Công Thành (E4), Backend (encoder cho câu thật)
report_feature_coverage.py	feature_coverage_report.json	Data report, review bàn giao

Con số phải thuộc lòng	Giá trị
Split / tổng	80,014 / 10,132 / 9,854 = 100,000
Evaluable / skipped	836 / 107 (ngưỡng positive 4.0)
Protocol	K = 10, 100 negative, seed 42
Year scaler (train-only, 1,611 phim)	median 1995.0, mean 1989.39, std 14.18
Hợp đồng feature	19 + 1 + 19 + 128 = 167 (hybrid-v1-167)
Pseudo-text	mean ≥ 4.0 và count ≥ 3, tối đa 3 genre; fallback 235/943 (24.92%)
Phim đặc biệt	1 thiếu năm; 71 cold-start; Nosferatu z = −4.75
Test	257 passed

Ba điều cấm với người tiêu thụ	Lý do một dòng
Không hardcode slice feature	Đọc ordered_feature_columns để đối hợp đồng thì code nổ chứ không âm thầm lệch
Không sinh lại feature khi train	Một nguồn sự thật duy nhất là artifact
Không quên lưu version theo checkpoint	Mọi con số phải truy ngược được về dữ liệu

16. Thuật ngữ Việt - Anh (bổ sung tuần tăng tốc)

Tiếng Việt	Tiếng Anh	Ghi nhớ
Hộ chiếu dữ liệu / bản kê	data manifest	Tự đếm, tự bấm, không tin số gõ tay
Dấu vân tay file	SHA-256 checksum	Đổi 1 byte → đổi cả chuỗi
Bàn giao đánh giá	evaluation handoff	seen / positive / candidate đóng gói thành file
Phim đã thấy	seen items	train u validation, loại khỏi bể negative
User chấm được / bị bỏ	evaluable / skipped users	836 / 107 — phải giống nhau giữa mọi model
Học tham số / áp dụng	fit / transform	Fit chỉ trên train
Điền giá trị thiếu	imputation (median)	Median 1995 học từ train
Điểm chuẩn hoá z	z-score	(x − mean) / std
Hồ sơ lịch sử thể loại	user genre history profile	mean của (r − 3)/2, thuộc [−1, 1]
Văn bản giả lập	pseudo-text	Sinh từ train, tuyên bố trung thực
Tích từng phần tử	Hadamard / element-wise product	u ⊙ m, giữ nguyên 128 chiều

Đường dự phòng	fallback	Luôn kèm cờ và con số
Đóng băng phiên bản	data freeze / version pinning	Đổi = bump version + chạy lại
Test canh giữ đóng băng	freeze-guard test	Pin giá trị config vào CI

17. Kết luận

Tuần 1 trả lời "dữ liệu có sạch và chia đúng không". Tuần tăng tốc trả lời ba câu khó hơn: **mô hình tiêu thụ dữ liệu qua hợp đồng nào** (167 chiều, ba nơi ghi cùng một thứ tự), **mọi so sánh dựa trên luật chung nào** (evaluation handoff với 836/107, K=10, seed 42), và **làm sao người khác tin được** (manifest checksum, 257 test, coverage report, freeze). Điểm bán của phần Data khi bảo vệ không phải là code nhiều — mà là *mọi con số trong báo cáo đều có đường truy ngược về một artifact có phiên bản, và mọi tính chất quan trọng đều có test cưỡng chế*. Nằm chắc mục 14 và cheat sheet mục 15 là đủ tự tin đứng trước mọi câu hỏi về dữ liệu.

Hết tài liệu — tiếp nối "Cẩm nang Data Pipeline" tuần 1 | [feature/data-hai-anh](#) | ml100k-temporal-v1